

# Table of Content

|                                                                                                                |          |
|----------------------------------------------------------------------------------------------------------------|----------|
| <b>A. Lab 1: Data Type Size Exploration.....</b>                                                               | <b>2</b> |
| Tasks:.....                                                                                                    | 2        |
| 1) Create a C program that prints the size of: char, int, float, double, long, unsigned<br>int, long long..... | 2        |
| 2) Compile and run on your system.....                                                                         | 3        |
| 3) Compare output with Table I.....                                                                            | 3        |
| 4) Why might sizeof(long) differ between 32-bit and 64-bit systems?.....                                       | 4        |
| <b>B. Lab 2: Memory Segment Address Mapping.....</b>                                                           | <b>5</b> |
| Tasks:.....                                                                                                    | 5        |
| 1) Write a program that declares variables in each segment:.....                                               | 5        |
| • Global initialized variable (Data segment).....                                                              | 5        |
| • Global uninitialized variable (BSS segment).....                                                             | 5        |
| • Local variable (Stack segment).....                                                                          | 5        |
| • Dynamically allocated variable (Heap segment).....                                                           | 5        |
| 2) Print the address of each variable using %p.....                                                            | 6        |
| 3) Identify which segment has the highest addresses.....                                                       | 6        |
| 4) Identify which segment has the lowest addresses.....                                                        | 6        |
| 5) Calculate the address difference between stack and heap.....                                                | 6        |
| <b>Knowledge Check Questions:.....</b>                                                                         | <b>7</b> |

## Table of Figure

|                                                           |   |
|-----------------------------------------------------------|---|
| Figure1: C program that prints the size of processes..... | 4 |
| Figure2: Compiled and run.....                            | 5 |
| Figure3: Comparing with Table 1.....                      | 5 |
| Figure4: Code snippet.....                                | 7 |
| Figure5: Compilation and running the code.....            | 8 |

## A.Lab 1: Data Type Size Exploration

### Tasks:

- 1) Create a C program that prints the size of: char, int, float, double, long, unsigned int, long long

```
ubuntu@LAPTOP-MBK6BRKM:~/batch38$ mkdir lab4
ubuntu@LAPTOP-MBK6BRKM:~/batch38$ cd lab4
ubuntu@LAPTOP-MBK6BRKM:~/batch38/lab4$ nano datatype.c
ubuntu@LAPTOP-MBK6BRKM:~/batch38/lab4$ cat datatype
cat: datatype: No such file or directory
ubuntu@LAPTOP-MBK6BRKM:~/batch38/lab4$ cat datatype.c
#include <stdio.h>

int main() {
printf("=== Data Type Sizes on This System ===\n\n");

printf("char: %zu bytes\n", sizeof(char));
printf("int: %zu bytes\n", sizeof(int));
printf("float: %zu bytes\n", sizeof(float));
printf("double: %zu bytes\n", sizeof(double))
;
printf("long: %zu bytes\n", sizeof(long));
printf("pointer: %zu bytes\n", sizeof(unsigned int));
printf("long long: %zu bytes\n", sizeof(long long));
return 0;
}
```

*Figure1: C program that prints the size of processes*

## 2) Compile and run on your system

```
ubuntu@LAPTOP-MBK6BRKM:~/batch38/lab4$ gcc datatype.c -o datatype
ubuntu@LAPTOP-MBK6BRKM:~/batch38/lab4$ ./ datatype
-bash: ./: Is a directory
ubuntu@LAPTOP-MBK6BRKM:~/batch38/lab4$ ./datatype
=== Data Type Sizes on This System ===

char: 1 bytes
int: 4 bytes
float: 4 bytes
double: 8 bytes
long: 8 bytes
pointer: 4 bytes
long long: 8 bytes
ubuntu@LAPTOP-MBK6BRKM:~/batch38/lab4$ |
```

Figure2: Compiled and run

## 3) Compare output with Table I

TABLE I  
DATA TYPE SIZES ACROSS SYSTEM ARCHITECTURES

| Data Type | 16-bit  | 32-bit  | 64-bit  | OS Impact                       |
|-----------|---------|---------|---------|---------------------------------|
| char      | 1 byte  | 1 byte  | 1 byte  | Consistent across architectures |
| short     | 2 bytes | 2 bytes | 2 bytes | Memory-efficient counters       |
| int       | 2 bytes | 4 bytes | 4 bytes | Affects pointer arithmetic      |
| long      | 4 bytes | 4 bytes | 8 bytes | Critical for 64-bit addressing  |
| long long | 8 bytes | 8 bytes | 8 bytes | Large file offsets              |
| float     | 4 bytes | 4 bytes | 4 bytes | IEEE 754 standard               |
| double    | 8 bytes | 8 bytes | 8 bytes | Precision calculations          |
| pointer   | 2 bytes | 4 bytes | 8 bytes | Address space limit             |

```
ubuntu@LAPTOP-MBK6BRKM:~/batch38/lab4$ gcc datatype.c -o datatype
ubuntu@LAPTOP-MBK6BRKM:~/batch38/lab4$ ./ datatype
-bash: ./: Is a directory
ubuntu@LAPTOP-MBK6BRKM:~/batch38/lab4$ ./datatype
=== Data Type Sizes on This System ===

char: 1 bytes
int: 4 bytes
float: 4 bytes
double: 8 bytes
long: 8 bytes
pointer: 4 bytes
long long: 8 bytes
ubuntu@LAPTOP-MBK6BRKM:~/batch38/lab4$ |
```

Figure3: Comparing with Table 1

Summary: The result from the code and Table 1 matches perfectly, except point where in the tabel 1 shows 8bytes for 64-bit system, but in my process it shows 4 bytes due to the use of the command (sizeof(int)) instead of (sizeof(int\*))

#### 4) Why might sizeof(long) differ between 32-bit and 64-bit systems?

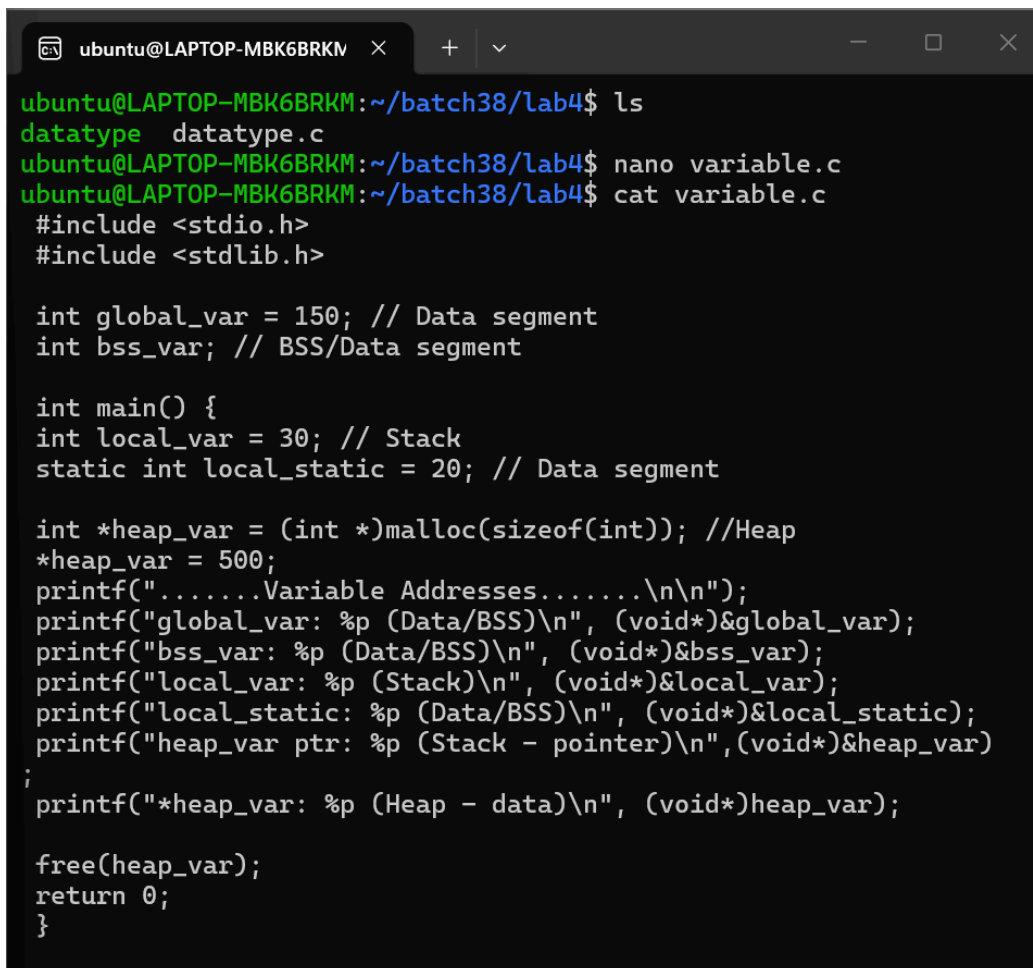
Ans. sizeof(long) differs between 32-bit and 64-bit systems due to the hardware CPU register and pointer size of the system. If CPU registers are 32 bits wide, then long is allocated 4bytes. But standard Linux/Unix systems expand pointers and long integers to 8 bytes to match the native 64-bit architecture and address space.

## B. Lab 2: Memory Segment Address Mapping

### Tasks:

1) Write a program that declares variables in each segment:

- Global initialized variable (Data segment)
- Global uninitialized variable (BSS segment)
- Local variable (Stack segment)
- Dynamically allocated variable (Heap segment)



```
ubuntu@LAPTOP-MBK6BRKM x + v - □ ×
ubuntu@LAPTOP-MBK6BRKM:~/batch38/lab4$ ls
datatype  datatype.c
ubuntu@LAPTOP-MBK6BRKM:~/batch38/lab4$ nano variable.c
ubuntu@LAPTOP-MBK6BRKM:~/batch38/lab4$ cat variable.c
#include <stdio.h>
#include <stdlib.h>

int global_var = 150; // Data segment
int bss_var; // BSS/Data segment

int main() {
int local_var = 30; // Stack
static int local_static = 20; // Data segment

int *heap_var = (int *)malloc(sizeof(int)); //Heap
*heap_var = 500;
printf(".....Variable Addresses.....\n\n");
printf("global_var: %p (Data/BSS)\n", (void*)&global_var);
printf("bss_var: %p (Data/BSS)\n", (void*)&bss_var);
printf("local_var: %p (Stack)\n", (void*)&local_var);
printf("local_static: %p (Data/BSS)\n", (void*)&local_static);
printf("heap_var ptr: %p (Stack - pointer)\n", (void*)&heap_var)
;
printf("*heap_var: %p (Heap - data)\n", (void*)heap_var);

free(heap_var);
return 0;
}
```

Figure4: Code snippet

2) Print the address of each variable using %p

```
ubuntu@LAPTOP-MBK6BRKM:~/batch38/lab4$ gcc variable.c -o variable
ubuntu@LAPTOP-MBK6BRKM:~/batch38/lab4$ ./variable
.....Variable Addresses.....

global_var: 0x600ebedc6010 (Data/BSS)
bss_var: 0x600ebedc601c (Data/BSS)
local_var: 0x7ffcdaa1304c (Stack)
local_static: 0x600ebedc6014 (Data/BSS)
heap_var ptr: 0x7ffcdaa13050 (Stack - pointer)
*heap_var: 0x600eccbe8010 (Heap - data)
ubuntu@LAPTOP-MBK6BRKM:~/batch38/lab4$ |
```

*Figure5: Compilation and running the code*

3) Identify which segment has the highest addresses

Ans. We can verify the highest addresses by how long or large as per the decimal. So, by doing that, we can see that the Stack segment has the highest addresses, which are local\_var ( 0x7ffcdaa1304c)and heap\_var ptr 0x7ffcdaa13050).

4) Identify which segment has the lowest addresses

Ans. The segment that has the lowest addresses is (Data/BSS), which is global\_var compared to other segments.

5) Calculate the address difference between stack and heap

Ans. Solution

Stack (0x7ffcdaa1304c) - heap (0x600eccbe8010) = 35,107,295,637,5 bytes

The difference between the stack and the heap is 35.1 terabytes of virtual memory addresses.

## Knowledge Check Questions:

1) What is a data type in the context of programming and operating systems?

Ans. In the context of programming and operating systems, A data type is a contract between the programmer, compiler, and OS that specifies.

- a. The bytes to be allocated in memory
- b. Interpret those bytes
- c. Valid operations on data

2) Name the five main memory segments in a process memory layout.

Ans. The five main memory segments are Stack, Heap, BSS, Data, and Text.

3) What is the difference between the Data segment and the BSS segment?

Ans. The Data segment holds global and static variables that the programmer explicitly initializes. The BSS segment holds global and static variables that are uninitialized, which the OS automatically zeros out upon execution.

4) Why does the OS place the stack at high virtual addresses and the text segment at low addresses?

Ans. The OS places the stack at high virtual addresses and the text segment at low addresses because the stack grows downward and the heap grows upward, which maximizes the flexible space between them and helps prevent memory overlap, collisions, and security issues.

5) What happens to stack memory when a function returns? Who reclaims it?

Ans. When a function returns, its stack frame is invalidated by moving the stack pointer back. The compiler generates the cleanup logic, and the OS reclaims it.

6) How does the OS know which memory belongs to which process?

Ans. The OS uses virtual memory management and hardware-level page tables, which are managed by the memory management unit. Each process gets its own private page table and is isolated in its address space, so it cannot view or modify memory allocated to another process.

Github link:

[https://github.com/santoshtula123/batch38\\_ai\\_os\\_santoshtuladhar/tree/main/lab4](https://github.com/santoshtula123/batch38_ai_os_santoshtuladhar/tree/main/lab4)